上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

计算机工程实践

章鸣铭
brucechang@sjtu.edu.cn

饮水思源 · 爱国荣校



Chapter 2

饮水思源 · 爱国荣校

机器狗运动控制

核心内容：

1. 运动库开发基础：讲解机器狗运动库架构、核心API接口及调用逻辑，适配图形化/Python编程。

2. 基础运动实现：实操调用运动库控制关节运动，调试步幅、步频等参数，实现前进、转向、姿态调整等基础动作，完成运动功能测试与优化。

01

机器狗基础运动控制

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

内容简介

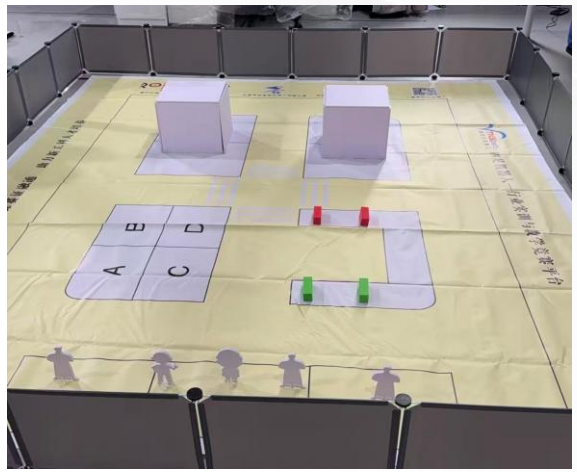
运动控制简介

- 通过对机器狗各个关节和步态的精确控制，机器狗能够完成**行走**、**转向**、**站立**、**趴下**等多种基础动作，并在复杂环境中保持平衡与稳定。
- 在本课程中，我们将使用 Python 编程对其运动控制接口进行学习与实践，逐步理解四足机器人运动的基本原理，包括**关节控制**、**姿态调整**和**步态运动**等内容。









机器狗运动库

初始化及机器狗型号判断

- 在VScode中创建一个新的Python文件，例如Test.py
- XGO内置了运动控制库文件xgolib.py开发者可以直接调用相关接口函数来控制机器狗。
- 首先导入库：

  from xgolib import XGO
- 之后进行机器狗的初始化：

  dog = XGO("xgolite")
- 在终端中运行python Test.py，查看输出结果

```python
#导入xgolib
from xgolib import XGO
#实例化dog
dog = XGO("xgolite")
```

```python
version = dog.read_firmware()
if version[0] == 'M':
    print('XGO-MINI')
else:
    print('XGO-LITE')
```

机器狗运动库

移动控制指令

- 机器狗运动库中封装了一系列方法，其中 move(),forward(),back(),left(),turn(), 等分别控制机器狗的前进，后退，左右平移以及转向。
- 通过x,y参数设置方向，通过step步长参数控制移动的速率，从而实现对机器狗运动的精准操控。

方法名	说明
move_x(step)	前后移动，相当于move('x', step)
move_y(step)	左右移动，相当于move('y', step)
forward(step)	前进，相当于move('x', abs(step))
back(step)	后退，相当于move('x', -abs(step))
left(step)	左移，相当于move('y', abs(step))
right(step)	右移，相当于move('y', -abs(step))
turnleft(step)	左转，相当于turn(abs(step))
turnright(step)	右转，相当于turn(-abs(step))

机器狗运动库

前后及左右移动控制

- 使用 move() 方法控制机器狗前后移动，其中x代表前进或后退，y代表左右平移。
- step数字代表迈步距离，正值代表向左及向前，负值代表向右及向后。注意步长范围不要超过限制范围。

move(direction, step)

参数名	格式	输入范围	说明
direction	字符	'x'、'X'、'y'、'Y'	'x'或'X'使机器狗前进或后退，'y'或'Y'使机器狗左移或者右移
step	数字	x:[-25,25],y:[-18,18]	该参数代表平移步长，根据方向，正值代表前进或左移，负值代表后退或右移。输入值超过范围时，按照极限值移动。

```python
# 导入XGO库
from xgolib import XGO
# 导入时间库
import time

# 初始化机器狗对象
dog = XGO("xgolite")

# 机器狗以步长20前进
dog.move('x', 20)
# 运动两秒
time.sleep(2)

# 机器狗以10步长向右平移
dog.move('y', -10)
# 运动两秒
time.sleep(2)
```

机器狗运动库

旋转控制

- 使用 turn () 指令控制机器狗旋转。step参数控制机器狗旋转速率，正值代表向左，负值代表向右。
- 如turn(90) 即为机器狗以90°/s 的速度向左旋转，此速度参数仅供参考，实际环境中会受摩擦力等系数的影响。

turn(step)

参数名	格式	输入范围	说明
step	数字	[-150,150]	该参数代表旋转速度，单位为°/s，正值为左转，负值为右转。

```python
# 导入XGO库
from xgolib import XGO
# 导入时间库
import time
# 初始化机器狗对象
dog = XGO("xgolite")

# 机器狗以90°/s左转
dog.turn(90)
time.sleep(2)

# 机器狗以90°/s右转
dog.turn(-90)
time.sleep(2)
```

机器狗运动开发

机器狗舵机控制

- 单腿控制 leg(leg_id, data):
- 控制指定腿的足端位置

参数名	格式	输入范围	说明
leg_id	整数	1、2、3、4	分别代表左前腿、右前腿、右后腿、左后腿
data	长度为3的数字列表	x:[-35,35],y:[-18,18],z:[75,115]	该参数代表足端位置，单位为mm

```
#设置右前腿足端坐标为(10mm,-12mm,90mm)
dog.leg(2,[10,-12,90])
```

机器狗运动开发

机器狗舵机控制

- 单腿舵机卸载 unload_motor(leg_id)：
- 使一条腿上的三个舵机卸载，不输出力矩，之后可以随意用手转动，一般用于编写动作。
- 所有舵机卸载 unload_allmotor()：
- 使所有舵机卸载，不输出力矩，可以随意用手转动

参数名	格式	输入范围	说明
leg_id	整数	1,2,3,4	分别代表左前腿、右前腿、右后腿、左后腿

机器狗运动开发

机器狗舵机控制

- 单腿舵机加载 load_motor(leg_id)：
- 使一条腿上的三个舵机保持当前位置加载，输出力矩，之后不可以用手转动，一般用于编写动作。
- 所有舵机加载 load_allmotor()：
- 使所有舵机加载，输出力矩，机器狗回到默认站姿，之后不可以用手转动

参数名	格式	输入范围	说明
leg_id	整数	1,2,3,4	分别代表左前腿、右前腿、右后腿、左后腿

机器狗运动库

结合时间控制的运动（函数定义）

- 使用 time.sleep() 指令控制机器狗执行命令的时间，单位为秒。使用 stop() 指令控制机器狗结束运动进程。
- 自行定义一个函数——move_and_turn()，使机器狗向前走出一段距离后，再掉头走并返回原点。

- 从程序设计的角度看，它有以下效果：
- 模块化与组织：它将完成“往返移动”这个完整任务的逻辑集中在一处，使代码结构更清晰、更易于维护。如果需要修改往返的流程，只需修改此函数一处。
- 提供可重复执行的单元：通过调用 move_and_turn()，可以随时、多次执行这组固定的操作序列，避免了代码的重复编写。

```python
# 导入XGO库
from xgolib import XGO
# 导入时间库
import time
# 初始化机器狗对象
dog = XGO("xgolite")

def move_and_turn():
    # 前进100mm
    dog.move('x', 100)
    time.sleep(2)

    # 旋转180°，即调头
    dog.turn(180)
    time.sleep(2)

    # 掉头后前进100mm返回原点
    dog.move('x', 100)
    time.sleep(2)

    #停止运动
    dog.stop()

# 调用函数
move_and_turn()
```

02

机器狗高级运动控制

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

高级运动控制

学习目标：

掌握机器狗的高级运动控制技能。

理解Python中的循环在运动控制中的应用。

本小结和以后的实验中，有一些新的注意事项：

- 在实验前确保机器狗**电量充足**。
- **实验中要注意机器狗的运动范围，避免碰撞或跌落。**
- 在使用循环和条件判断时，确保逻辑正确，**避免无限循环。**
- **如果陷入死循环中，应先用Ctrl+C及时中断程序，并用软急停脚本 stop.py 及时终止机器狗。**

高级运动控制

原地踏步控制

使用 mark_time() 指令控制机器狗进行原地踏步，step 参数控制踏步的姿态。

▾ **原地踏步，只适用于mini**

mark_time(data)

参数名	格式	输入范围	说明
data	数字	[10,35]	该参数代表原地踏步抬腿高度，单位为mm，输入为0时停止原地踏步

```python
# 导入XGO库和时间模块
from xgolib import XGO
import time
# 初始化机器狗对象
dog = XGO("xgolite")

# 机器狗原地踏步，抬腿高度为20mm
dog.mark_time(20)
# 踏步持续3秒
time.sleep(3)

# 停止原地踏步
dog.reset()
```

高级运动控制

迈步频率控制

- 使用 pace() 指令控制机器狗运动过程中的迈步频
- 通过快中慢三档不同的频率进行调节。

pace(mode) 速度 = 步频 x 步幅

参数名	格式	输入范围	说明
pace(mode)	字符串	['normal','slow','high']	该参数代表迈步频率，normal为默认步频，low为慢速步频，high为高速步频

```python
# 导入XGO库和时间模块
from xgolib import XGO
import time
# 初始化机器狗对象
dog = XGO("xgolite")

# 将迈步频率设置为慢速
dog.pace('slow')
# 机器狗前进50mm
dog.move('x', 50)
time.sleep(2)

# 恢复默认迈步频率
dog.pace('normal')
```

高级运动控制

运动控制结合循环

编写代码运行，使用循环结构控制机器狗进行多次移动。

这次我们引入了一个循环结构，这个循环结构实现了以下效果：

自动化重复执行：它将“前进-后退”这一对动作指令定义为一个可重复执行的标准单元，并通过for循环结构自动运行指定次数，避免了重复代码的冗余。

参数化与结构化控制：它将控制循环行为的关键要素提取为变量，使得整个运动模式易于调节和配置。

可重用的逻辑单元：这个模式可以被视为一个功能单元，在程序中只需要调整参数即可控制运动的幅度和次数，而不需重写运动逻辑，增强了代码的模块性和可维护性。

```python
# 导入XGO库和时间模块
from xgolib import XGO
import time
# 初始化机器狗对象
dog = XGO("xgolite")

# 机器狗前进和后退的循环
steps = 5  # 步数
step_length = 30  # 每步长度

for _ in range(steps):
    dog.move('x', step_length)
    time.sleep(1)
    dog.move('x', -step_length)
    time.sleep(1)
```

高级运动控制

结合条件判断的运动控制

通过检测某个条件（如时间或传感器数据），决定机器狗的运动。在传感器和姿态的内容中，我们将会学习设计check_condition()中的条件。

这段代码在框架上实现了：

条件驱动：通过while循环与条件检测结合，使运动持续执行直到条件满足，实现响应式控制。

逻辑分离：将条件判断封装为独立函数，使运动控制与条件检测分离，提高代码的适应性和可维护性。

持续行为：循环体将“前进-后退”定义为可重复执行的行为单元，适用于需要持续响应环境的任务场景。。

```python
# 导入XGO库和时间模块
from xgolib import XGO
import time
# 初始化机器狗对象
dog = XGO("xgolite")

#check_condition()来检测是否满足条件
def check_condition():
    # 这里应该是条件检测的代码
    return True

# 机器狗根据条件判断进行运动
while not check_condition():
    dog.move('x', 10)  # 前进
    time.sleep(0.5)
    dog.move('x', -10)  # 后退
    time.sleep(0.5)
    print('Keep moving!')
```

03

位姿控制综合应用

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

准备工作

学习目标

掌握机器狗位姿控制的高级技巧。

学习如何将位姿控制与Python高级编程技巧结合使用。

理解并应用面向对象编程(OOP)的概念来控制机器狗。

注意事项

确保树莓派和机器狗的电源充足。

实验中要注意机器狗的运动范围，避免碰撞或跌落。

在编写面向对象的代码时，注意方法和属性的封装。

位姿控制

位姿相关方法

机身位置平移 translation(direction, data)

参数名	格式	输入范围	说明
direction	单字符或字符列表	'x'、'y'、'z'或包含以上值的列表	'x'代表前后平移，'y'代表左右平移，'z'代表身高
data	数字	x:[-35,35],y:[-18,18],z:[75,115]	该参数代表机身位置平移距离，单位为mm

机身姿态调整 attitude(direction, data)

参数名	格式	输入范围	说明
direction	单字符或字符列表	'r'、'p'、'y'或包含以上值的列表	'r'代表滚转角，'p'代表俯仰角，'y'代表偏航角
data	数字	r:[-20,20],p:[-15,15],y:[11,11]	该参数代表机身姿态调节幅度，单位为°

位姿控制

位姿相关方法

机身周期旋转 periodic_rot(direction, period)

参数名	格式	输入范围	说明
direction	单字符或字符列表	'r'、'p'、'y'或包含以上值的列表	'r'代表滚转角，'p'代表俯仰角，'y'代表偏航角
period	数字	[1.5,8]	该参数代表运动周期，单位为s;输入0时代表停止运动

位姿控制

位姿相关方法

机身周期平移 periodic_tran(direction, period)

机器狗机身将以指定周期和方向进行往复平移，幅度为位置平移极限值的一半，可以同时进行多个方向的周期运动。机身周期运动和整机运动不可同时进行。

参数名	格式	输入范围	说明
direction	单字符或字符列表	'x'、'y'、'z'或包含以上值的列表	'x'代表前后平移，'y'代表左右平移，'z'代表高低移动
period	数字	[1.5,8]	该参数代表运动周期，单位为s;输入0时代表停止运动

位姿控制

定义机器狗类

在这里，我们声明了一个“类”RobotDog。类可以有成员函数，例如这里的 translate() 函数。

类（Class）是Python面向对象编程的核心模块单元，主要有三大作用：

封装：将数据与相关操作绑定为独立对象，对外隐藏实现细节。

代码组织：将相关的属性和方法组织在统一命名空间下，使代码结构清晰、模块化，便于维护和扩展。

状态管理：通过实例属性（如self.dog）保存对象状态，确保方法间可共享和操作同一数据，保持功能一致性。

```python
class RobotDog:
    def __init__(self, model):
        self.dog = XGO(model)

    def translate(self, direction, distance):
        # 机身位置平移
        if direction == 'x':
            self.dog.translation('x', distance)
        elif direction == 'y':
            self.dog.translation('y', distance)
        # 等待移动完成
        time.sleep(0.5)

    def adjust_attitude(self, roll, pitch, yaw):
        # 机身姿态调整
        self.dog.attitude(['r', 'p', 'y'], [roll, pitch, yaw])
        # 等待姿态调整完成
        time.sleep(0.5)

    def periodic_motion(self, direction, period):
        # 机身周期运动
        if direction == 'x':
            self.dog.periodic_tran('x', period)
        elif direction == 'y':
            self.dog.periodic_tran('y', period)
        # 持续一定时间后停止
        time.sleep(period * 2)
        self.dog.periodic_tran(direction, 0)
```

位姿控制

创建机器狗对象并执行位姿控制

- 实例化机器狗对象，并调用方法进行位姿控制。
- 上段代码定义了类，但是还没有对其进行实例化，是没有运行效果的。这次运行程序，我们能看到机器狗的移动。

```
# 创建XGO-LITE机器狗对象
my_dog = RobotDog("xgolite")

# 机身向前平移50mm
my_dog.translate('x', 50)
# 机身向右平移25mm
my_dog.translate('y', 25)
#机身俯仰角设置为10°
#偏航角设置为-4°
#滚转角设置为8°
my_dog.adjust_attitude(8, 10, -4)

# 机身以3秒的周期，进行前后往复平移
my_dog.periodic_motion('x', 3)
```

位姿控制

创建机器狗对象并执行位姿控制

- 在Python中，**__init__是一个特殊方法（构造函数），在创建对象时自动执行，用于初始化对象的状态。**
- 我们可以在类当中继续声明方法，这被叫做**类的成员函数。**成员函数很**特殊，**它们有以下**特点**：
- 第一个参数通常是self，指向对象实例本身，用于访问和操作对象的属性
- 可以访问和修改对象的状态（如self.dog）
- 封装了与对象相关的操作逻辑

这是在 Python 中面向对象的典型应用。

我们首先声明了一个**类：通过class RobotDog:语句**，我们声明了一个名为RobotDog的类。**这个类作为一个模板**，定义了机器狗这类对象应该具有的数据（属性）和行为（方法）。

```
#类的声明
class RobotDog:
    def __init__(self, model):
        self.dog = XGO(model)

#RobotDog类的一个成员函数
def adjust_attitude(self, roll, pitch, yaw):
    # 机身姿态调整
    self.dog.attitude(['r', 'p', 'y'], [roll, pitch, yaw])
    # 等待姿态调整完成
    time.sleep(0.5)
```

位姿控制

创建机器狗对象并执行位姿控制

创建类的实例：通过类名(参数)的形式创建类的具体实例。创建了RobotDog类的一个实例my_dog，并传递型号参数"xgolite"给构造函数__init__。每个实例都拥有独立的属性空间，但共享类定义的方法。

调用成员函数：通过实例.方法名(参数)的语法调用成员函数。在这些调用中，Python自动将my_dog实例作为self参数传递给方法，使方法能够操作特定实例的状态。

```
# 创建XGO-LITE机器狗对象
my_dog = RobotDog("xgolite")
```

类的实例

```
# 机身以3秒的周期，进行前后往复平移
my_dog.periodic_motion('x', 3)
```

调用成员函数

04

机械臂控制

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

机械臂相关方法

设置机械臂末端位置arm(arm_x, arm_z)

参数名	格式	输入范围	说明
arm_x	float	[-80, 155]	单位为mm
arm_z	float	[-95, 155]	单位为mm

- 此处的x和z是相对于机械臂的基座的坐标，单位为毫米。
- 设定超过机械臂工作空间的值时，机械臂会保持最后一个有效值对应的姿态，比如(155,0)对应的姿态是向前伸到最大，(0,155)对应向上伸到最大，(155,155)是斜向上最大。当机械臂达不到预设位置时，就会保持上一次发送的有效位置。

机械臂相关方法

设置机械臂夹爪开合 claw(pos)

参数名	格式	输入范围	说明
pos	uint_8	0-255	0对应完全张开，255对应完全闭合

设置机械臂是否开启稳定模式 arm_mode(mode)

参数名	格式	输入范围	说明
mode	int	0\1	0不开启，1开启

开启之后机械臂末端会不随着身体的平移而平移（平移指四脚站定躯干运动，而非前后左右迈步平移）

机器狗运动开发

机械臂相关方法

创建一个类来封装机器狗机械臂的控制方法

预设机械臂的安全工作范围：

- arm_x：-80 ~ 155 mm（水平面左右 / 前后移动极限）；
- arm_z：-95 ~ 155 mm（垂直面上下移动极限，-95为最低极限，对应桌面抓取）；

超出该范围时，不执行移动操作，仅打印错误提示，避免机械臂过载或卡死，保护硬件设备；

```python
from xgolib import XGO
import time

class RobotDogArm:
    def __init__(self):
        self.dog = XGO("xgolite")

    def move_arm(self, arm_x, arm_z):
        """
        控制机械臂移动到指定位置。

        :param arm_x: 水平面移动距离(mm)
        :param arm_z: 垂直面移动距离(mm)
        """
        # 检查输入值是否在机械臂工作范围内
        if -80 <= arm_x <= 155 and -95 <= arm_z <= 155:
            self.dog.arm(arm_x, arm_z)
        else:
            print("Error: Arm position out of range.")

    def control_claw(self, pos):
        """
        控制机械臂夹爪的开合。

        :param pos: 夹爪开合位置(0-255)
        """
        self.dog.claw(pos)
```

机器狗运动开发

机械臂相关方法

- 按顺序执行「移动机械臂→夹爪开到中间→夹爪完全张开→夹爪完全闭合」的一系列动作。
- 调用time.sleep(2)，暂停 2 秒，给机械臂足够时间完成移动动作（硬件动作响应慢于代码执行速度，延时是嵌入式硬件控制的必备操作）。

```
# 创建机器狗机械臂对象
my_arm = RobotDogArm()

# 移动机械臂到指定位置
my_arm.move_arm(20, 30)  # 例如：向前移动20mm，向上移动30mm

time.sleep(2)

# 控制夹爪开合
my_arm.control_claw(128)  # 例如：夹爪开合至中间位置
time.sleep(2)

# 夹爪完全张开
my_arm.control_claw(0)
time.sleep(2)

# 夹爪完全闭合
my_arm.control_claw(255)
time.sleep(2)
```

机器狗运动开发

机械臂相关方法

在控制机械臂时，使用异常处理来捕获并处理可能发生的错误。

常规方法可用try except捕获异常：

```python
# 尝试移动机械臂到超出工作范围的位置
try:
    my_arm.move_arm(200, 200)  # 这将超出机械臂的工作范围
except Exception as e:
    print(f"An error occurred: {e}")
```

可用条件判断先避免了出现错误的可能，也是程序控制中常见的一种避免出现错误的方法。

```python
def move_arm(self, arm_x, arm_z):
    """
    控制机械臂移动到指定位置。

    :param arm_x: 水平面移动距离(mm)
    :param arm_z: 垂直面移动距离(mm)
    """
    # 检查输入值是否在机械臂工作范围内
    if -80 <= arm_x <= 155 and -95 <= arm_z <= 155:
        self.dog.arm(arm_x, arm_z)
    else:
        print("Error: Arm position out of range.")
```

课后实践

- 修改RobotDog类，添加新的方法来控制机器狗的身高变化。
- 编写一个脚本，使机器狗执行一个复杂的动作序列，如先向前移动，然后调整姿态，最后进行周期性运动。

05

舵机角度控制（选学）

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

舵机编号

这对应机器狗身体上的15个舵机，其具体编号可以在舵机贴的标签处查看，也可对应表格。

部位	1.末端	2.臂身	3.基座
1.左前腿	11	12	13
2.右前腿	21	22	24
3.右后腿	31	32	33
4.左后腿	41	42	43
5.机械臂	51（夹爪）	52（小臂）	53（大臂）

```
FULL_MOTOR_ACTION = [
    # 帧1: 初始站立 + 机械臂收回
    {'duration': 10.0, 'mode': MODE_MOTOR, 'data': [
        # 腿部12个（标准站立），机械臂3个
        0, 56, 0,   0, 56, 0,   0, 56, 0,   0, 56, 0,  0, 0, -50
    ]},
```

舵机控制

腿部舵机

1号舵机控制小腿，数值为0时代表垂直于大腿，正值向前，负值向后。

2号舵机控制大腿，数值为0时代表垂直与机身，正值向后，负值向前。

3号舵机控制基座，数值为0时代表平行与地面，正值向外旋转，负值向内旋转。

机械臂舵机

1号舵机控制夹爪，正值收缩，负值扩张。

2号舵机控制小臂，数值为0是代表平行于地面，正值向下，负值向上。

3号舵机控制大臂，数值为0时垂直于地面，正值向前负值向后。

舵机控制示例

通过精准设置每个舵机的动作与运动时间，可以实现复杂的动作控制

```
"Pee": [
    {'duration': 1.5, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, -0.1, -21.0, 90.0, -0.1, -21.0, 90.0, -0.1, -21.0, 90.0, -0.1, 0]},
    {'duration': 1.5, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -1.4, 68.7, -4.4, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, 3.5, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -11.2, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, 3.5, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -11.2, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, 3.5, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 1.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -11.2, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, 3.5, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -11.2, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 0.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, 3.5, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
    {'duration': 1.2, 'mode': MODE_MOTOR, 'data': [-21.0, 90.0, 4.1, -1.4, 68.7, -4.4, -11.2, 68.7, -30.0, -21.0, 90.0, 4.1, 0]},
],
```

```
"Find_Food": [
    # Data format: x, y, z, rol, pit, yaw, vx, vy, vyaw, T, ...
    # 注意: xgolib 中 translation/attitude 只有 3 个参数，我们需要拆分 data
    {'duration': 0.5, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 20, 0, 0, 0, 0]},
    {'duration': 0.3, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 20, 15, 0, 0, 0]},
    {'duration': 0.1, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 10, 15, 0, 0, 0]},
    {'duration': 0.1, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 20, 15, 0, 0, 0]},
    {'duration': 0.1, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 10, 15, 0, 0, 0]},
    {'duration': 0.1, 'mode': MODE_BODY, 'data': [15, 0, 75, 0, 20, 15, 0, 0, 0]},
    {'duration': 2.5, 'mode': MODE_BODY, 'data': [0, 0, 75, 0, 5, 0, 8, 0, 0]},
]
```



谢谢！

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center